NIRMAN Data Automation — Architecture Analysis & Migration Plan

Current Architecture

Structure

The existing system is a **single-purpose workshop rating extractor** with 3 main files:

File	Purpose	Lines
	Core extraction, validation, Excel export (monolith)	532
	Flask web UI + API routes + state management	263
	Single-page UI with inline CSS/JS	639

What Works Well (Preserving)

- ✅ Two-pass extraction (messy-field detection → value extraction)
- ✅ Sentinel values: `ILLEGIBLE`, `FIELD_MISSING`, `OUT_OF_RANGE`, `API_ERROR`, `PARSE_ERROR`
- ✅ Retry logic with exponential backoff for 503/429
- ✅ Color-coded Excel output
- ✅ Live progress polling + per-form status badges
- ✅ Session persistence via `results.json`
- ✅ Manual edit → re-export workflow
- ✅ `.env` file loading for API keys

What Must Change

- ❌ Hard-coded `Q1-Q30` rating fields — not dynamic
- ❌ No sample analysis — form structure is assumed
- ❌ No schema management — single form type only
- ❌ Only supports numeric 1-5 scale ratings — no text, checkbox, date, categorical

- ❌ No analytics or report generation
- ❌ No validation layer beyond range checks
- ❌ No PDF support
- ❌ No test suite
- ❌ No duplicate filename handling
- ❌ Raw/validated value distinction missing
- ❌ No confidence tracking
- ❌ Prompt hardcodes Devanagari/workshop context

New Architecture

```
nirman/
├── __init__.py
├── config.py                # Centralized config (env vars, paths, limits)
├── models.py                # Pydantic data models (schemas, records, results)
├── ai_provider.py           # Gemini API wrapper with retry logic
├── sample_analyzer.py       # Analyze sample form → generate schema
├── schema_manager.py        # CRUD for form schemas (load, save, list, validate)
├── extractor.py             # Schema-driven extraction pipeline
├── validator.py             # Field validation (type-aware, sentinel handling)
├── excel_export.py          # Professional Excel workbook generation
├── analytics.py             # Statistical analysis of batch data
├── report_generator.py      # AI-powered report generation
├── audit.py                 # Audit logging (privacy-aware)
├── file_handler.py          # Upload validation, duplicate handling, PDF support
├── app.py                   # Flask app (routes only, thin controller)
├── templates/
│   └── index.html           # Redesigned multi-step UI
├── tests/
│   ├── __init__.py
│   ├── test_models.py
│   ├── test_sample_analyzer.py
│   ├── test_schema_manager.py
│   ├── test_extractor.py
│   ├── test_validator.py
│   ├── test_excel_export.py
│   ├── test_analytics.py
│   └── test_report_generator.py
├── schemas/                 # Saved form schemas (JSON files)
├── uploads/                 # Uploaded form images (batch processing)
├── output/                  # Generated Excel/report files
├── requirements.txt
├── start.py
├── Start NIRMAN.bat
├── start.sh
└── README.md
```

Migration Plan

Stage 1: Core Models & Config

Create `nirman/` package with `config.py` (env loading, paths) and `models.py` (Pydantic schemas for `FieldDefinition`, `FormSchema`, `ExtractionRecord`, `BatchResult`, `ValidationResult`).

Stage 2: AI Provider & File Handling

Extract Gemini client + retry logic into `ai_provider.py`. Create `file_handler.py` for upload validation, duplicate filenames, image/PDF support.

Stage 3: Sample Analyzer & Schema Manager

`sample_analyzer.py` sends a sample form to AI and generates a `FormSchema`. `schema_manager.py` saves/loads/lists schemas.

Stage 4: Schema-Driven Extraction & Validation

`extractor.py` uses the schema to build dynamic prompts. `validator.py` handles type-aware validation (numeric, text, checkbox, date, categorical). Preserves raw + validated values.

Stage 5: Excel Export & Analytics

`excel_export.py` generates multi-sheet workbooks. `analytics.py` computes statistics, distributions, summaries.

Stage 6: Report Generation

`report_generator.py` uses AI to generate plain-language analytical reports from validated data.

Stage 7: Flask App & UI Redesign

`app.py` becomes a thin controller. `index.html` gets a multi-step wizard workflow.

Stage 8: Tests

Comprehensive test suite for all modules.

Stage 9: Documentation & Cleanup

Updated README, setup instructions, example workflows.

> IMPORTANT
>
> The old `workshop_extractor.py` and `app.py` will be archived as `_legacy/` but their functionality is fully preserved and enhanced in the new modules.

Breaking Changes

1. **Directory structure**: Code moves into `nirman/` package
2. **Form schema required**: Must analyze a sample before batch processing (new step, but enables any form type)
3. **Output location**: Excel/reports go to `output/` directory
4. `results.json` **format**: New format includes schema reference, raw/validated values, confidence
5. **API routes**: All routes change to support multi-step workflow